

Paradigma P2P

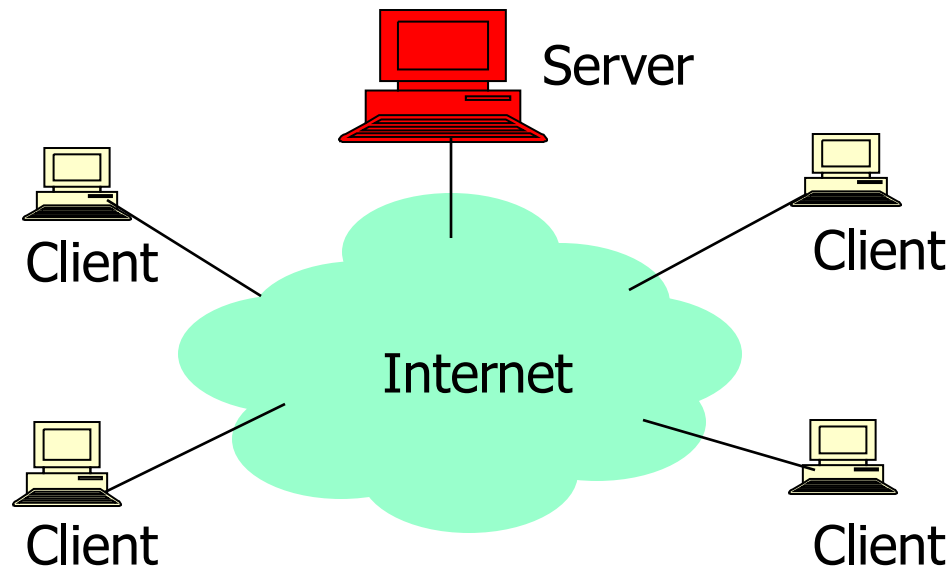
Lenuta Alboaie (lenuta.alboaie@info.uaic.ro)
Andrei Panu (andrei.panu@info.uaic.ro)

Cuprins

- **Paradigma peer-to-peer (P2P)**
 - Preliminarii
 - Definitii
 - Caracterizare
 - Tipuri de aplicatii
 - Infrastructuri
 - Instrumente

Preliminarii

...sa ne reamintim **modelul client/server**



Preliminarii

...sa ne reamintim **modelul client/server**

- Uzual, privim clientul ca fiind o componenta avand capacitati computationale reduse
- Serverul este mentinut si administrat in mod centralizat

Probleme ale arhitecturii client/server:

- Lipsa robustetii
- Lipsa sigurantei (eng. *reliability*)
- Lipsa scalabilitatii
- Vulnerabilitate la atac

Definitii

Peer = *one that is of equal standing with another*
(conform Webster)

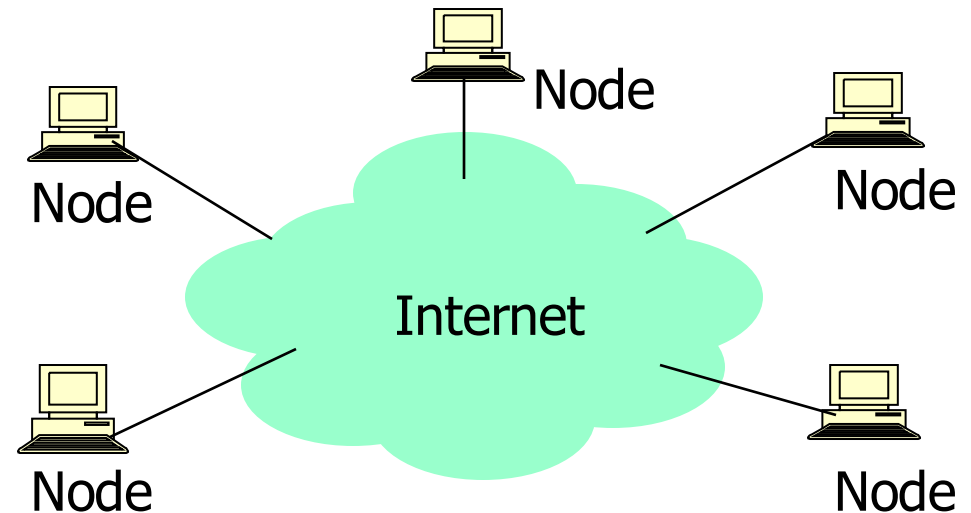
Peer-to-peer (P2P) = arhitectura de retea in care
nodurile sunt relativ egale

- In sensul ca fiecare nod este, in principiu, capabil sa realizeze functii specifice retelei

Definitii

Sistemele P2P, in sens strict, sunt sisteme complet distribuite

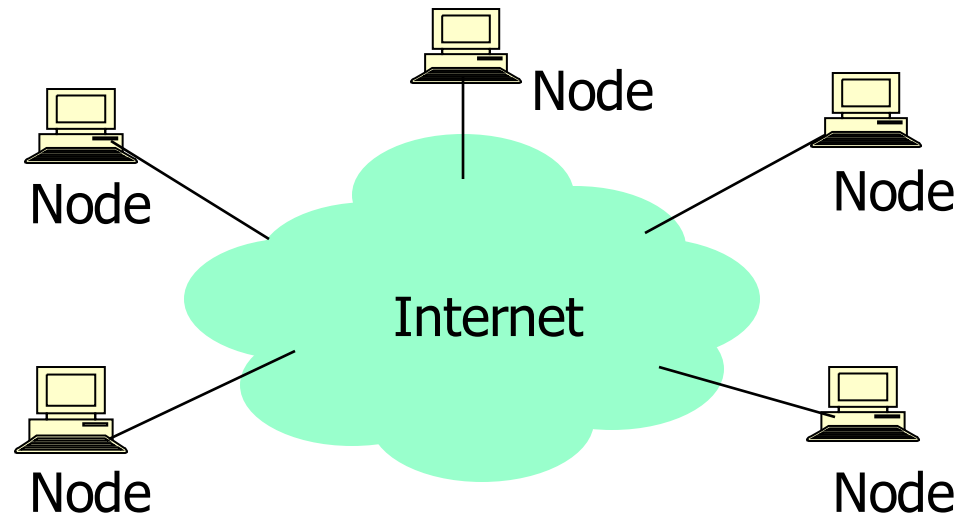
- Toate nodurile sunt total echivalente, in termeni de functionalitate si a activitatilor pe care le pot desfasura



Obs.: **Sistemele P2P pure** sunt rare (de ex. Gnutella); majoritatea sunt hibride, avand supernoduri sau servere cu diferite roluri (de ex. cautare de date, control etc.)

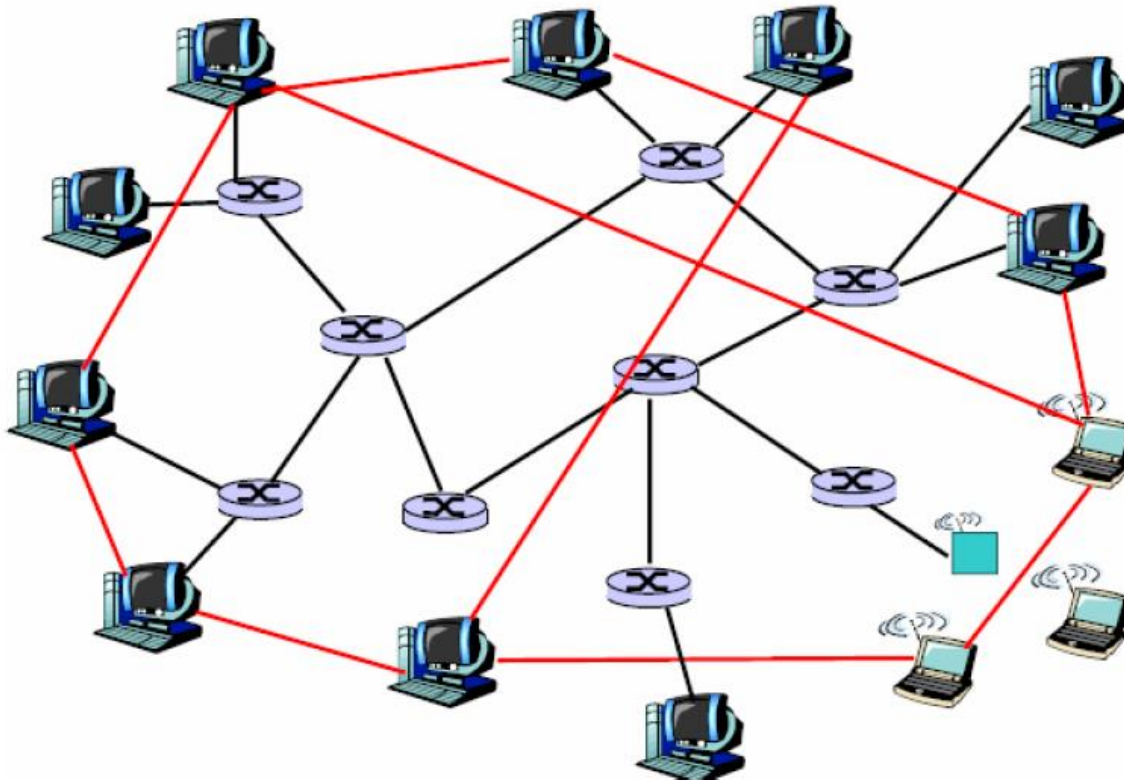
Definitii

- Nodurile
 - Pot consuma si oferi date
 - Orice nod poate initia o conexiune
- Nu exista sursa de date centralizata =>
 - Forma de democratie pe Internet
 - Protectie *copyright* amenintata



Definitii

- “**P2P**” este clasa de aplicatii care se bazeaza pe resursele (de stocare, de procesare, continut, prezente umane) disponibile la marginile (*edges*) Internet-ului



*Edges of the Internet
(overlay networks)*

Definitii

“**P2P** is a class of applications that take advantage of resources – storage, cycles, content, human presence – available at the edges of the Internet. Because accessing these decentralized resources means operating in an environment of unstable and unpredictable IP addresses, P2P nodes must operate outside the DNS system and have significant or total autonomy from central servers”

“A distributed network architecture may be called a **P2P** network if the participants share a part of their own resources. These shared resources are necessary to provide the service offered by the network. The participants of such a network are both resource providers and resource consumers”

Caracterizare

Caracteristici definitorii:

- Partajarea resurselor computationale prin interschimb **direct** si mai putin prin intermediari oferite de o autoritate centralizata (server)
 - Serverele centralizate pot fi folosite insa pentru a realiza activitati specifice (initializarea retelei P2P, adaugarea de noi noduri in retea,...)
 - Ideal, nodurile participa activ si unilateral la realizarea de operatii ca localizarea & caching-ul nodurilor/continutului, dirijarea informatiilor, managementul resurselor transferate etc.

Caracterizare

Caracteristici definitorii:

- Abilitatea de a trata instabilitatea si variatiile conctivitatii retelei, **adaptandu-se automat** la erorile survenite sau la dinamicitatea nodurilor
 - Topologia retelei P2P e adaptiva si toleranta la defecte, nodurile auto-organizandu-se in vederea mentinerii conectivitatii si performantei retelei

Caracterizare

Reteaua P2P este una suprapusa (*overlay*) peste cea fizica

- Se situeaza la nivelul aplicatie => flexibilitate
- Muchiile virtuale sunt conexiuni TCP sau pointeri la adrese IP
- Mentinerea rețelei P2P se face prin verificarea periodica a conectivitatii (*ping*) ori a existentei (mesaje “*still alive?*”)
- Cand un nod pica, sistemul P2P ar putea stabili noi muchii
- Proximitatea (fizica) a nodurilor nu e importanta
- Reteaua P2P poate fi structurata sau nu

Scopuri si beneficii

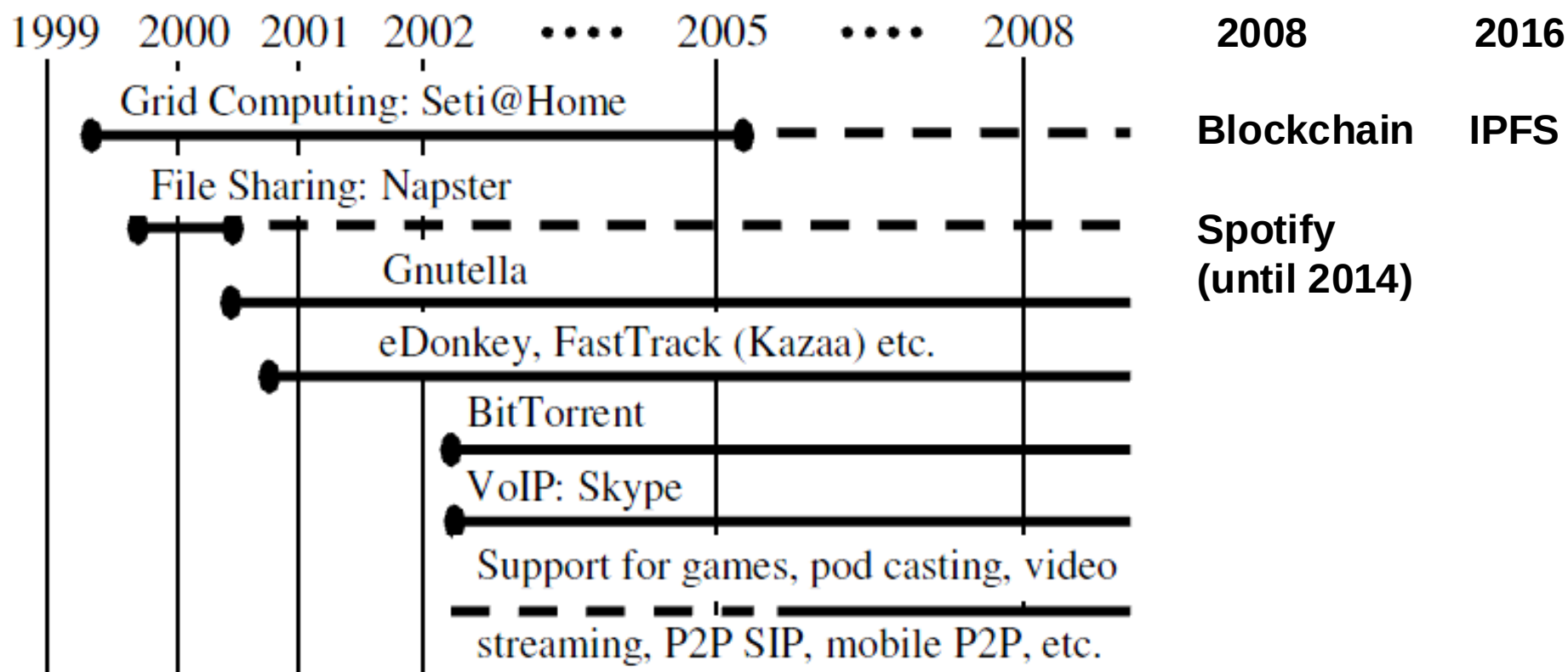
- **Utilizarea eficienta a resurselor**
 - Latimea de banda neutilizata, resurse de stocare, putere de procesare disponibile la marginile (*edges*) rețelei
- **Scalabilitate**
 - Fara informatii centralizate, fara *bottleneck*-uri (de comunicare si de calcul)
 - Agregarea resurselor se face in mod natural odata cu utilizarea sistemului
- **Siguranta** (eng. *reliability*)
 - Existenta de copii a datelor
 - Distribuire geografica
 - Nu mai exista “*single point of failure*”
- **Administrare usoara**
 - Nodurile se auto-organizeaza
 - Cresterea tolerantei la erori si a echilibrarii incarcarii
 - Cresterea autonomiei
- **Anonimitatea**
 - Greu de realizat intr-un sistem centralizat
- **Dinamism**
 - Mediu dinamic
 - Colaborare si comunicare ad-hoc

Dezavantaje/Probleme

- Arhitecturile P2P sunt probabilistice
 - Localizare impredictibila a resurselor
 - Resursele sunt volatile
- Inexistenta unui control centralizat
 - Probleme privind impunerea unei autoritati asupra aplicatiilor, continutului si utilizatorilor
 - Dificultati in detectarea si identificarea utilizatorilor (aspecte anti-sociale)
- Incurajarea folosirii sistemelor P2P in scop abuziv si ilegal (de ex. drepturile de autor asupra continutului digital)
- Lipsa increderii la nivel comercial, de afaceri
- Probleme de securitate (curs viitor)

Evolutie...

Timeline of Popular Peer-to-Peer Protocols



Tipuri de aplicatii

- **Comunicare & colaborare**

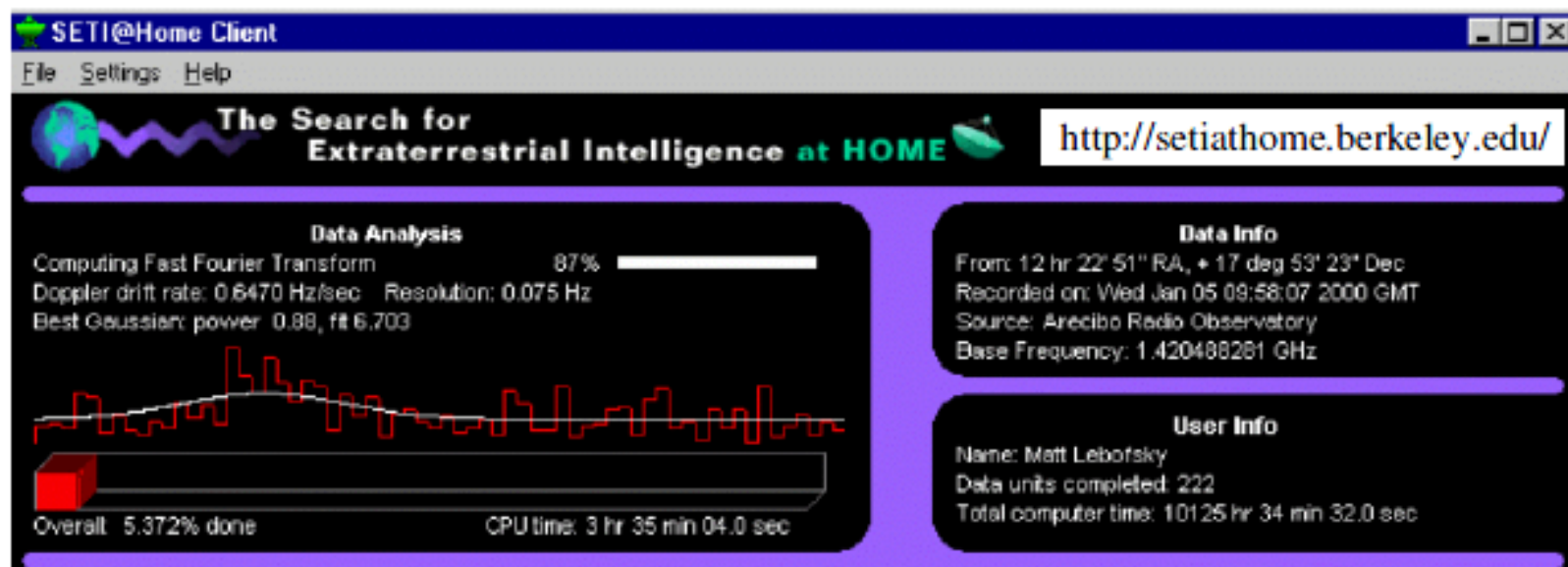
- Sisteme ce ofera o infrastruktura pentru facilitarea comunicarii & colaborarii directe, deseori in timp real, intre noduri
 - Sisteme conversationale (chat, mesagerie instantanee):
IRC (Internet Relay Chat), ICQ (1996), YM!, MSN Messenger, Skype, Sisteme multicast P2P (e.g. Cirrus – Adobe Flash <http://labs.adobe.com/technologies/cirrus/>; WebRTC), ...

- **Calcul distribuit**

- Sisteme ce folosesc puterea computationala a nodurilor disponibile (cicli de procesor)
 - Rezolvarea unor probleme prin *divide-et-impera*: SETI@home (*Search for Extra-Terrestrial Intelligence*-Berkeley), genome@home
 - Reteaua P2P reprezinta un gen de Grid computational (...curs master)

Tipuri de aplicatii | Calcul distribuit - Exemplu

SETI@Home: A Public-Resource Computing Experiment



- ❑ Radio telescope signal analysis has insatiable appetite for computing power
- ❑ Usage of computers in homes and offices around the world has provided unprecedented computing power
- ❑ Grid computing application via peer-to-peer approach under central control

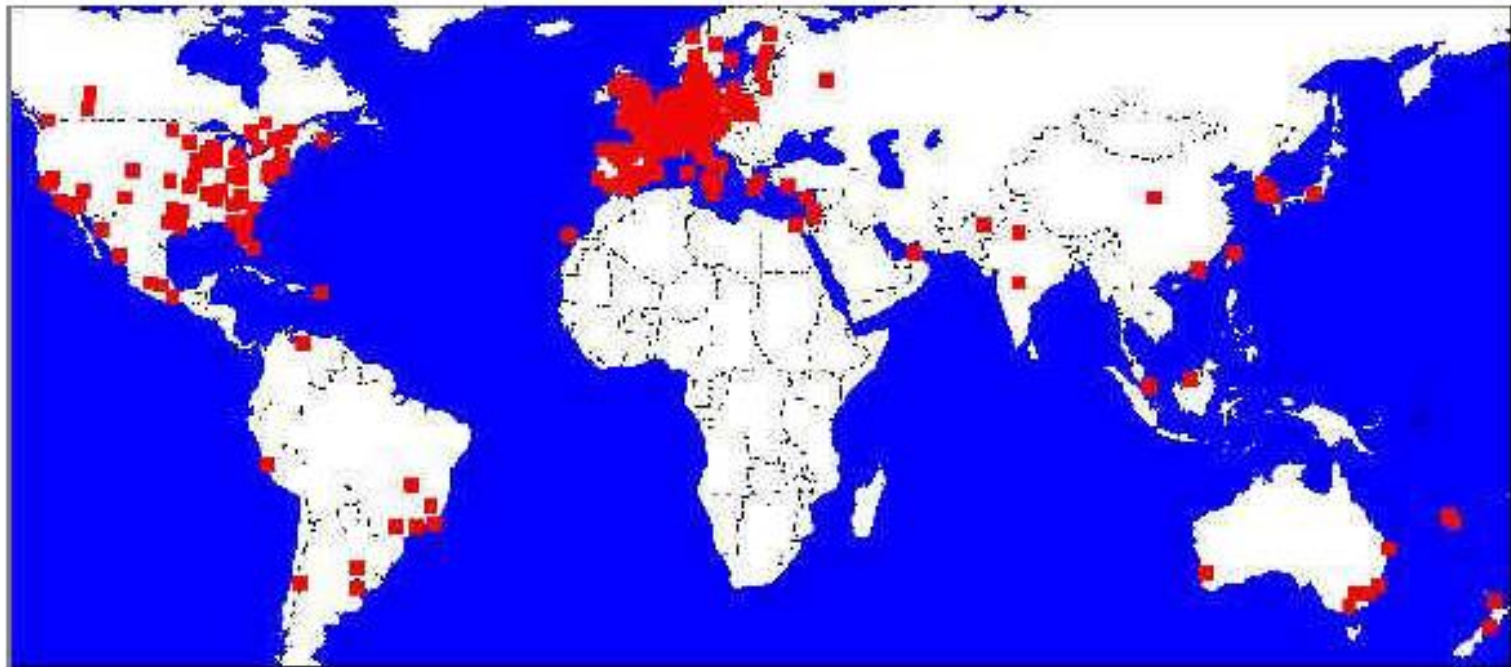
Tipuri de aplicatii

- **Sisteme de stocare (baze de date)**
 - Proiectare de sisteme de baze de date distribuite bazate pe infrastructuri P2P
 - PIER – motor scalabil de interogare distribuita (<https://pier.cs.berkeley.edu/>)
 - Edutella – proiect open-source pentru interogari si stocare de *meta-data* (P2P pentru Semantic Web)
- **Distribuire de continut digital**
 - Sisteme & infrastructuri pentru partajarea resurselor digitale (multimedia si alte date) intre utilizatori
 - Aplicatii pentru partajarea fisierelor (e.g. Napster, Gnutella, KaZaA, Freenet, BitTorrent, eDonkey etc.)
 - Medii de stocare distribuita pentru publicarea, organizarea, indexarea, cautarea si regasirea datelor in maniera securizata & eficienta
(PAST, Chord, Groove, Mnemosyne, Avalanche,...)

Tipuri de aplicatii

- **Distribuire de continut digital | Exemplu**

P2P File-Sharing: Fast distribution of large files



Example: Harry Potter III early propagation after 2 hours on May 28th 2004 (Source: www.itic.ca/DIC/News/archive.html)

Tipuri de aplicatii

- **Distribuire de continut prin P2P**
 - **Sisteme P2P de “interschimb de fisiere”**
 - In cazul unora, nodurile transfera un fisier la un moment dat
 - Se ofera facilitati pentru realizarea unei retele P2P si pentru cautarea si transferul de fisiere intre noduri
 - Nu se ofera suport pentru securitate, disponibilitate si persistenta
 - Exemple: **Napster, KaZaA, Gnutella, BitTorrent**

Tipuri de aplicatii

- **Distribuire de continut prin P2P**
 - Sisteme P2P pentru publicarea & stocarea continutului
 - Utilizatorii pot publica, stoca si distribui continut digital, pe baza unor drepturi de acces (privilegii)
 - Se focalizeaza asupra securitatii si persistentei
 - Unele ofera si facilitati privind colaborarea intre utilizatori
 - Exemple: Scan, Groove, Freenet, MojoNation, Tangler

Tipuri de aplicatii

- **Distribuire de continut prin P2P**
 - **Infrastructuri** pentru:
 - **Dirijare & Localizare:**
Chord, Can, Pastry, Tapestry, Kademlia
 - **Anonimitate:**
Onion Routing, ZeroKnowledge, Freedom, Tarzan
 - **Managementul reputatiei:**
Eigentrust, PeerTrust

Infrastruc.(localizare & dirijare)

- Mecanismele de localizare si dirijare ce pot fi adoptate depind de:
 - Topologia
 - Structura
 - Gradul de centralizareale rețelei suprapuse, acoperitoare (*overlay network*)

Infrastruc.(localizare & dirijare)

- **Aspecte privind centralizarea**
 - **Arhitecturi pur descentralizate**: toate nodurile realizeaza exact aceleasi activitati, jucand simultan roluri de servere si clienti, fara a beneficia de o coordonare centrala
 - Nodurile se numesc **servents** (SERVers + clieENTS)

Infrastruc.(localizare & dirijare)

- **Aspecte privind centralizarea**
 - **Arhitecturi partial centralizate**: unele noduri au un rol mai important (de ex. stocand indecsi locali pentru fisierele partajate)
 - Nodurile devin **supernoduri** conform politicilor fiecarui sistem P2P
 - Rolul de supernod este stabilit dinamic
 - **Arhitecturi descentralizate hibride**: exista un server central facilitand interactiunea intre noduri, mentinand cataloage de meta-date ale fisiereilor
 - Serverele pot identifica si verifica nodurile de stocare
 - Sistemele se mai numesc *broker mediated*

Infrastruc.(localizare & dirijare)

- **Aspecte privind structura rețelei:**
 - **Nestructurata:** plasarea conținutului este complet independentă de topologia rețelei suprapuse
 - Conținutul trebuie localizat
 - Strategii de căutare prin “*forta bruta*”: inundarea rețelei – cereri propagate via BFS/DFS
 - Strategii mai sofisticate: drumuri aleatorii, probabilistice etc.
 - **Slab structurata** (*loosely structured*): deși localizarea conținutului nu e complet specificată, aceasta este afectată de dirijare
 - Categorie aflată între rețele structurate și cele nestructurate

Infrastruc.(localizare & dirijare)

- **Aspecte privind structura rețelei:**
 - **Structurata:** topologia este controlata, iar fisierele (sau pointerii la ele) sunt plasate in locatii precise
 - Se realizeaza o asociere (*mapping*) intre continut (identificatorul de fisier) si locatie (adresa nodului)
 - In genul unei tabele de rutare distribuita
 - Cautarile exacte (*exact-match queries*) pot fi realizate in mod scalabil
 - Structura folosita la dirijarea eficienta a mesajelor este dificil de mentinut in cazul unor noduri tranziente, cu rata mare de atasare si deconectare de la retea

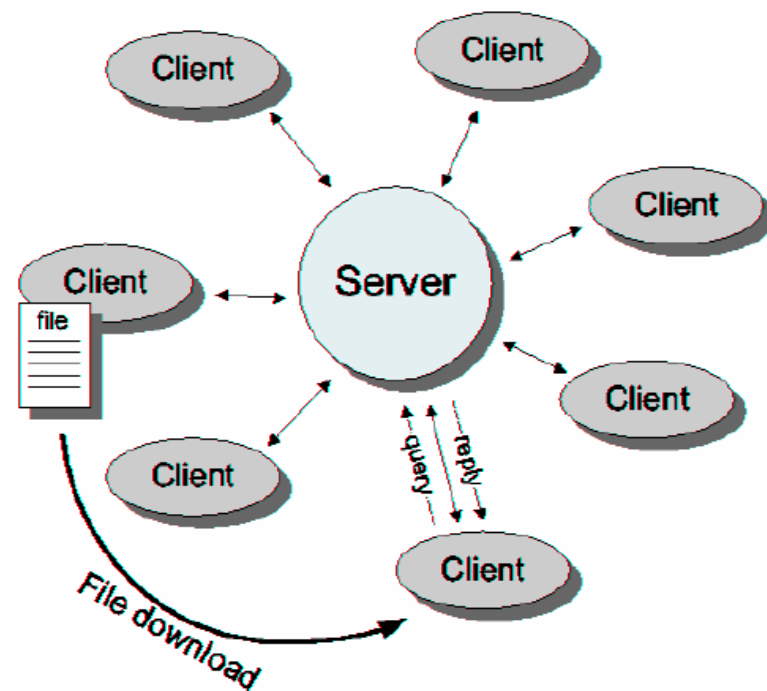
Infrastruc.(localizare & dirijare)

	Centralizare		
	Hibridă	Parțială	Absentă
Nestructurată	Napster Publius	BitTorrent KaZaA Morpheus Edutella	Gnutella FreeHaven
Infrastructură structurată			Chord, CAN, Tapestry, Pastry
Sisteme structurate			OceanStore Scan, PAST, Kademlia, Tarzan

Arhitecturi nestructurate

Descentralizate hibride

- Fiecare calculator client stocheaza continut (fisiere) partajat(e)
- Serverul central mentine o tabela cu conexiunile utilizatorilor inregistrati (IP, latime de banda,...) + o tabela cu lista fisierelor fiecarui utilizator & meta-date
- Exemple: **Napster, Publius**



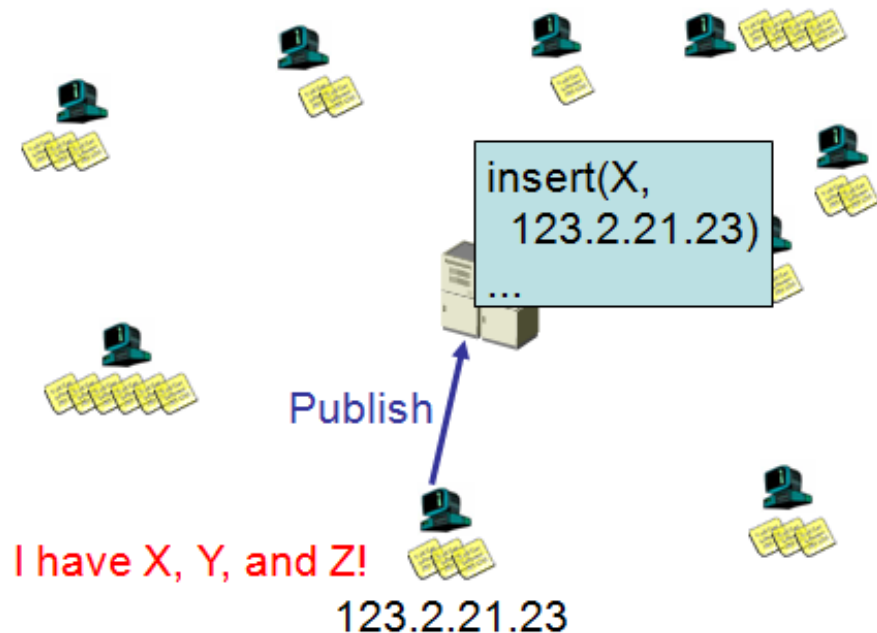
Arhitecturi nestructurate

Napster

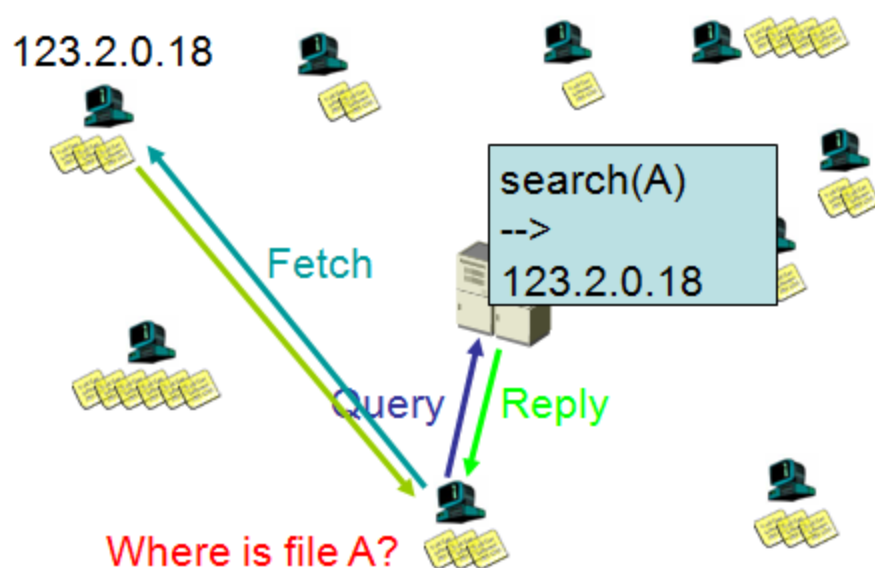
- 1999: Sean Fanning lanseaza Napster
- A atins cota de 1.5 milioane de utilizatori simultan
- Baza de date centralizata - operatii:
 - **Join**: clientul contacteaza serverul central (via TCP)
 - **Publish**: raportarea unei liste de fisiere serverului central
 - **Search**: interogarea serverului => se intoarce cineva care stocheaza fisierul cerut
 - **Fetch**: ia fisierul direct de la *peer* (cel cu cea mai buna rata de transfer)
- Iulie 2001: Napster a fost inchis

Arhitecturi nestructurate

Napster: Publish



Napster: Search



Discutii:

- Serverul face toate procesarile
- Avem "single point of failure"
- Probleme de scalabilitate, unele sisteme nu permit adaugarea altor servere (lista serverelor disponibile este statica)

Arhitecturi nestructurate

Descentralizate pure

- Se construiește o rețea acoperitoare (*overlay*) cu propriile mecanisme de rutare prin IP
- Nu există o coordonare centrală
- Utilizatorii se conectează via o aplicație care are rol dublu – *servent*
- Comunicarea între *serventi* se bazează pe un protocol la nivel de aplicație, cu 4 tipuri de mesaje:
 - Ping – cere ca un nod să se anunțe
 - Pong – replica la mesajul *ping* (IP, port, numărul & mărimea fișierelor)
 - Query – cerere de căutare (șir de căutare + viteză minimă de transfer)
 - Query hints – răspuns (IP, port, viteză, dim. fis., index fis.)

Arhitecturi nestructurate

Descentralizate pure

- Cautarea se realizeaza prin inundare (flooding)
 - Daca nu ai fisierul dorit, intreaba pe n vecini
 - Daca nici ei nu au fisierul, vor intreba pe vecinii lor in maxim m hop-uri
 - Pe calea de intoarcere se vor intoarce raspunsurile (nu continutul fisierelor)
- Fiecare mesaj are un TTL atasat
- Exemplu: **Gnutella**

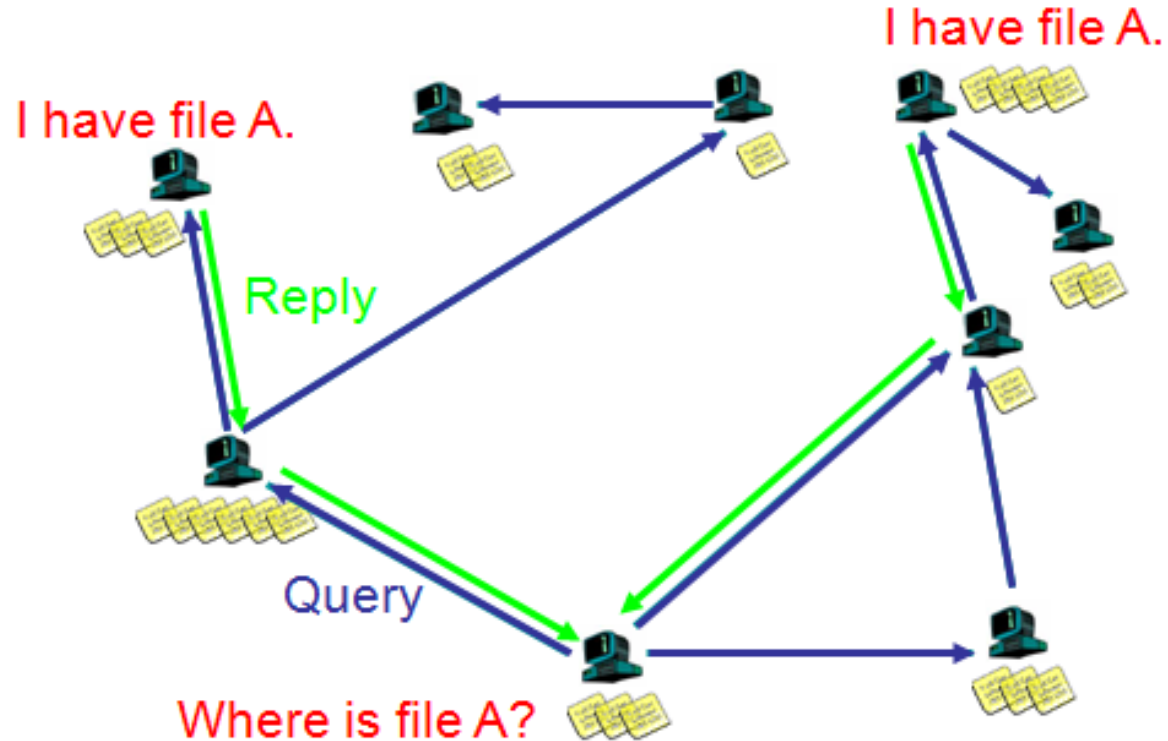
Arhitecturi nestructurate

Gnutella

- 2000: L. Frankel si T. Pepper (Nullsoft) lanseaza Gnutella
- Apar clienti: Bearshare, Morpheus, LimeWire
- *Query Flooding*:
 - **Join**: la intrare in sistem, clientul contacteaza cateva noduri care devin “vecinii” sai
 - **Publish**: nu este necesar
 - **Search**: se intreaba vecinii, care isi intreaba vecinii lor, ...
 - Exista un TTL ce limiteaza propagarea
 - **Fetch**: preia fisierul direct de la *peer*

Arhitecturi nestructurate

Gnutella



Aspecte:

- Timpul de cautare este... $O(?)$
- Nodurile pleaca adesea => retea instabila

Arhitecturi nestructurate

Partial centralizate

- Folosesc conceptul de **supernod**: are activitati de servire a unei sub-rețele P2P (indexare, *caching*)
- Nodurile sunt alese automat ca fiind supernoduri daca au suficienta latime de banda si putere computationala
- Toate cererile sunt trimise initial la supernoduri
- Avantaje: timpul descoperirii resurselor e mai redus + eterogenitatea este exploatata
- Exemplu: **KaZaA**

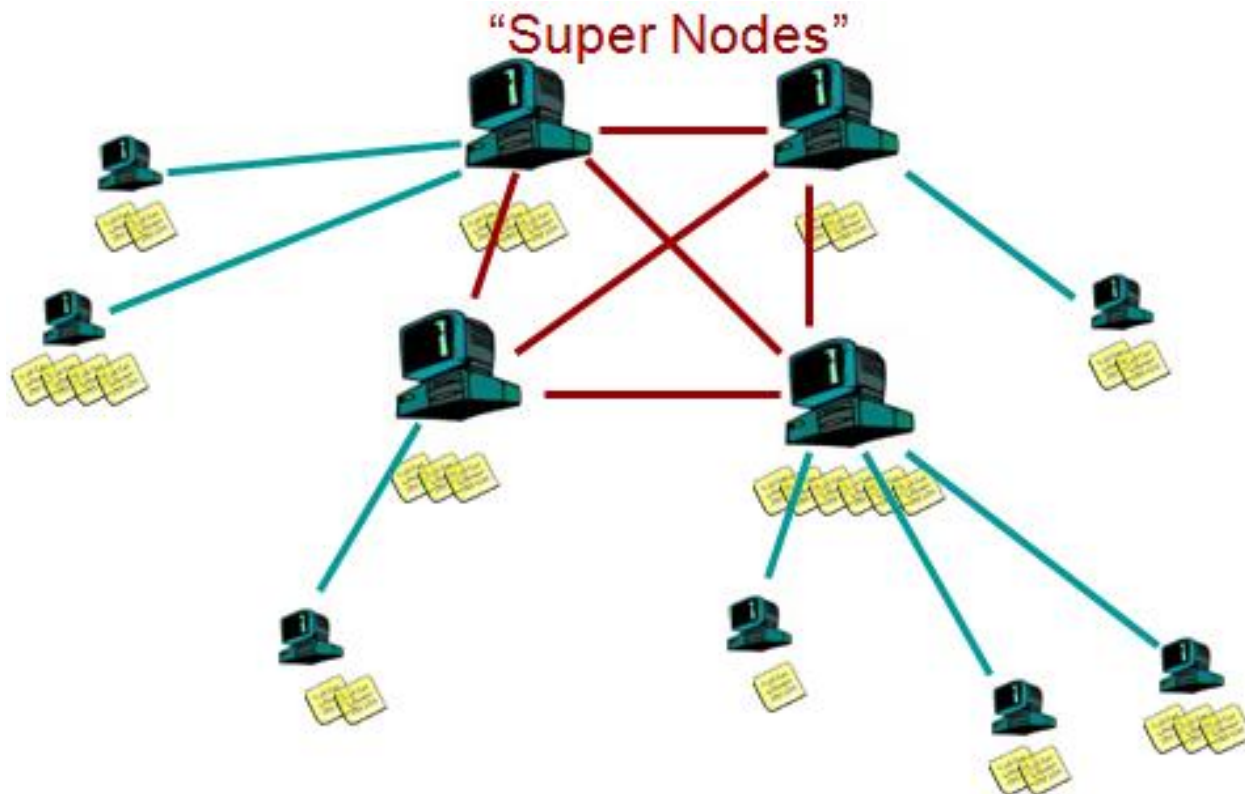
Arhitecturi nestructurate

KaZaA

- 2001: Se lanseaza KaZaA
- Apar clienti: Morpheus, giFT
- Se utilizeaza un mecanism de tip “*smart*” *query flooding*:
 - **Join**: la intrare in sistem, clientul contacteaza un “*supernode*” (poate deveni si el *supernod* la un moment dat)
 - **Publish**: trimite lista de fisiere *supernodului*
 - **Search**: trimite interogarea *supernodului*, si *supernodurile* se interogheaza intre ele
 - **Fetch**: ia fisierul direct de la *peer(s)*; poate prelua fisierul simultan de la mai multe *peer-uri*

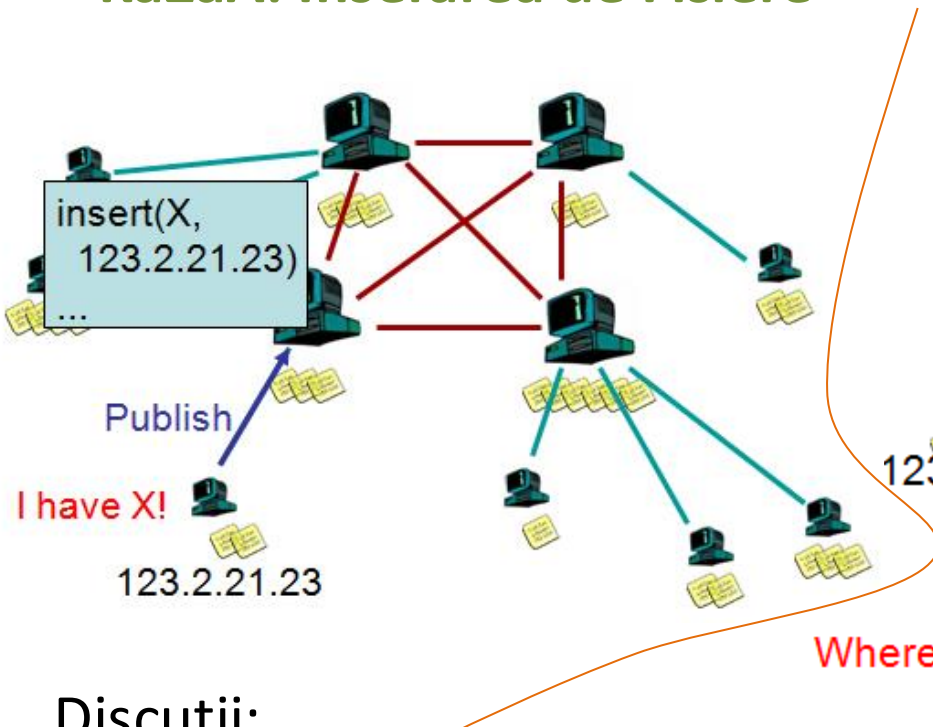
Arhitecturi nestructurate

KaZaA: Designul rețelei

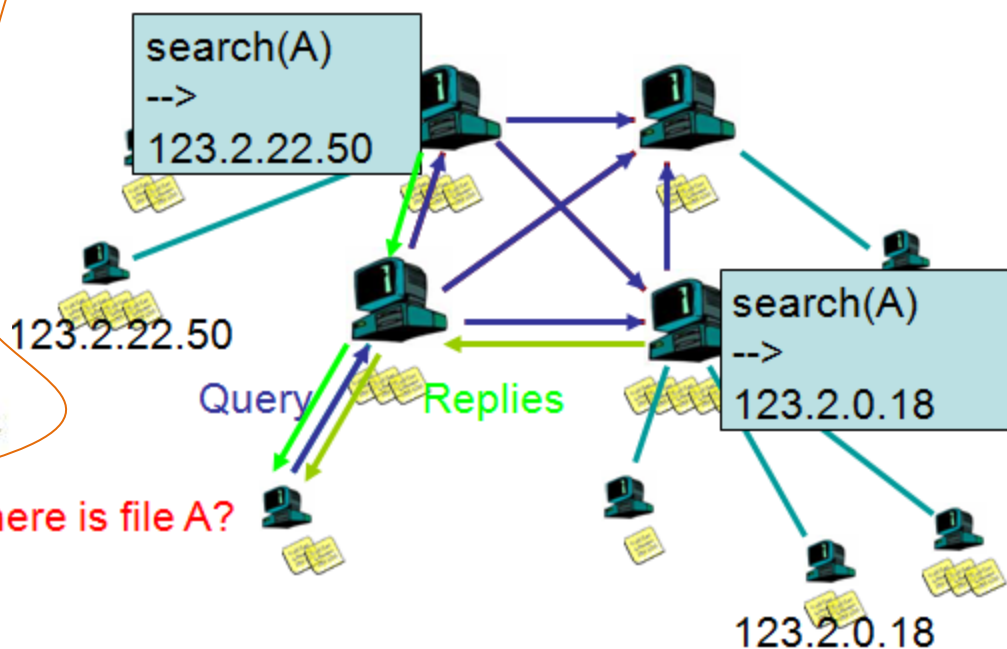


Arhitecturi nestructurate

KaZaA: Inserarea de Fisiere



KaZaA: Cautarea de Fisiere



Discutii:

- Comportament similar cu Gnutella, dar mai eficient
- Nu este nici o garantie asupra timpului de cautare sau a domeniului de cautare

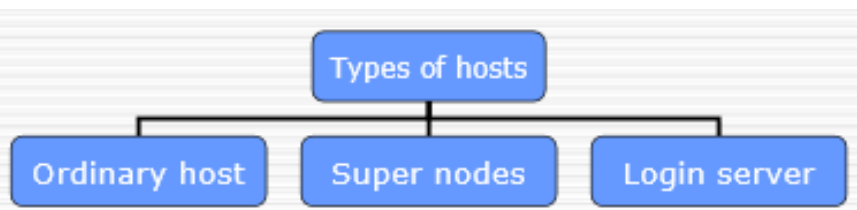
Arhitecturi nestructurate

Partial centralizate

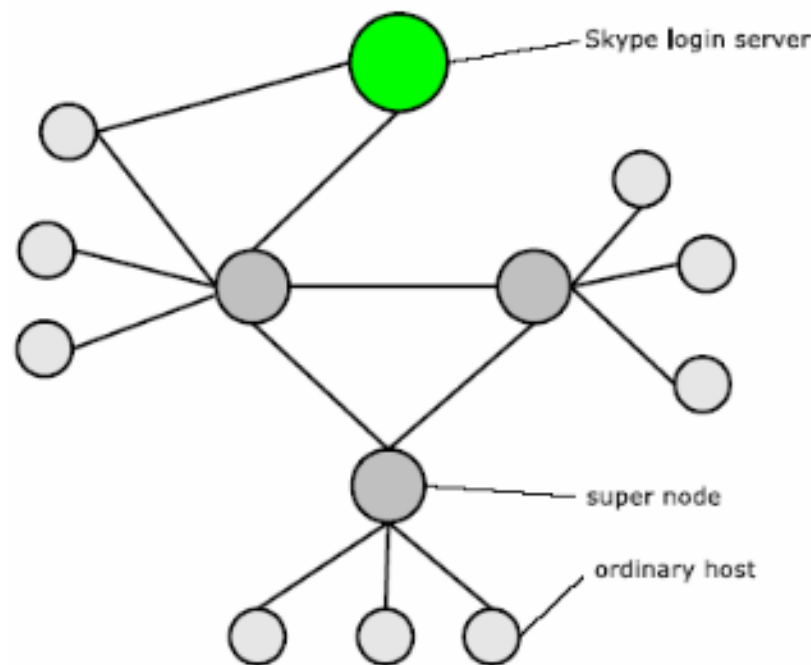
- Software-ul KaZaA este proprietar
- Datele de control P2P sunt criptate
- Mesajele folosesc HTTP
- Un nod e fie un supernod, fie asignat unui supernod
- Un supernod are 100-150 noduri-copil
- O retea poate avea ~30000 supernoduri
- Fiecare supernod are conexiuni TCP cu 30-50 supernoduri
- Pentru fiecare fisier se mentin meta-date (nume, dimensiune, *content hash*, descriptor de fisier)
- *Content hash*-ul este folosit pentru cautarea altei copii a unui fisier partial transferat
- Varianta fara *spyware* si *pop-up*-uri: **KaZaA-lite**

Arhitecturi nestructurate

Skype



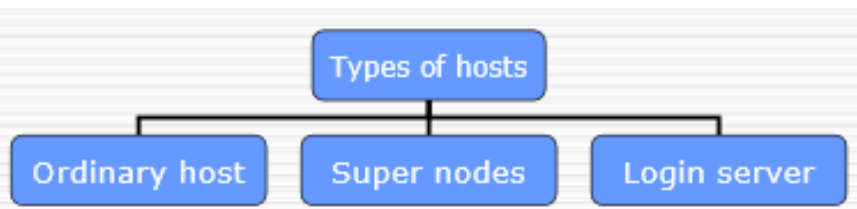
- Prima retea de telefonie P2P bazata pe IP
- din Iunie 2014, Microsoft a anuntat incompatibilitatea cu protocolul anterior Skype
- foloseste Microsoft Notification Protocol 24 (prima utilizare -> MSN Messenger in 1999)
- arhitectura era similara cu KaZaA



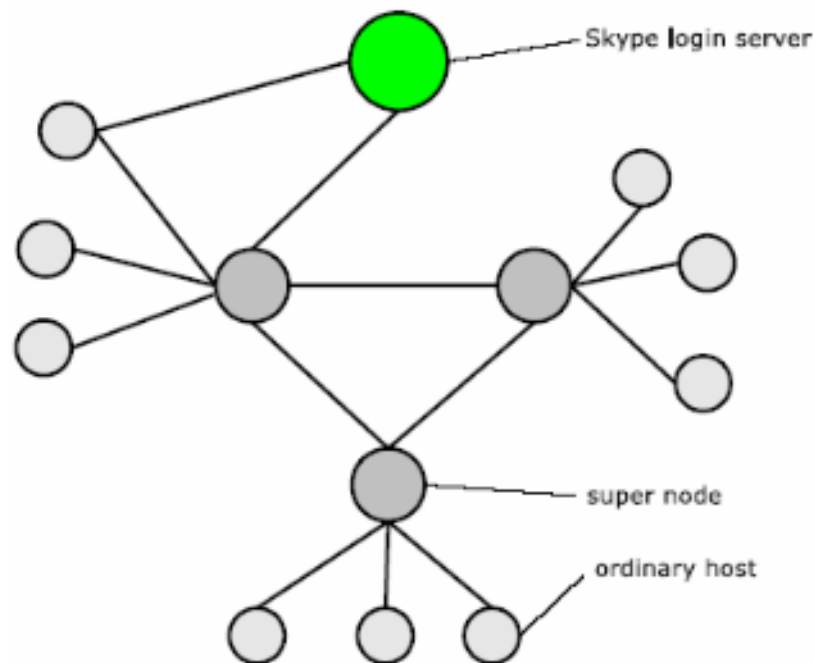
http://www1.cs.columbia.edu/~salman/publications/skype1_4.pdf

Arhitecturi nestructurate

Skype



- Fiecare client mentinea un *host cache* cu adresele IP si numerele de port ale supernodurilor accesibile
- Orice client cu latime de banda (si fara restrictii de firewall sau NAT) putea deveni supernode
- din 2012, Microsoft a inceput gazduirea supernodurilor in servere din centrele sale de date



<https://en.wikipedia.org/wiki/PRISM>

Arhitecturi nestructurate

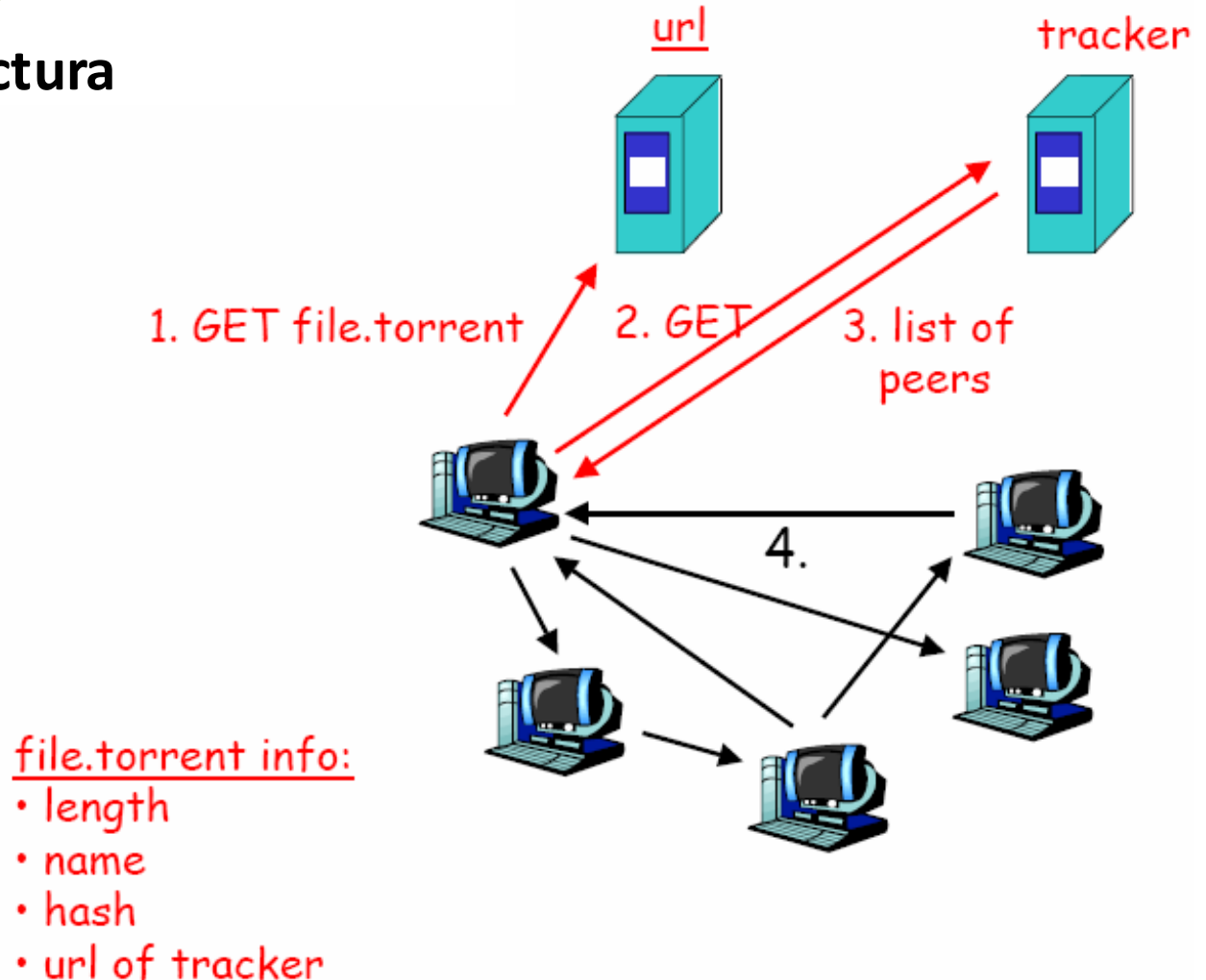
Partial centralizate

- Daca un fisier este gasit pe mai multe noduri, transferul poate fi realizat in paralel
 - Copiile identice se identifica via *content hash*
- Diferite portiuni din fisier sunt transferate de pe noduri diferite
- Pentru transferurile intrerupte, se face o recuperare automata (*automatic recovery*)
- Exemplu: **BitTorrent**
 - In 2002, B. Cohen a lansat BitTorrent
 - Si-a propus concentrarea pe problema legata de obtinerea eficienta a resurselor (*efficient fetching*) si nu pe cautare (*searching*)
 - Sustinatori inca de la aparitie
 - Blizzard Entertainment folosea BitTorrent pentru distributia versiunilor beta a noilor jocuri

Arhitecturi nestructurate

Partial centralizate

BitTorrent - arhitectura



Arhitecturi nestructurate

Partial centralizate

BitTorrent

- Se bazeaza pe mecanismul de *swarming*:
 - **Join**: contacteaza un server centralizat (*tracker*) si obtine o lista de *peer-uri*
 - **Publish**: ruleaza un server *tracker*
 - **Search**: de ex. foloseste motor de cautare pentru a gasi un *tracker* pentru fisierul dorit
 - **Fetch**: Preia bucati de fisiere de la *peer-uri*; incarca bucatile de fisier pe care le ai
- Obs. Diferente fata de Napster:
 - *Download-ul* de bucati (*chunk*) de fisiere
 - Utilizarea strategiei “tit-for-tat”: daca A face *download* de la alte noduri, atunci A trebuie sa permita si *download-ul* de la el (*free-rider problem*)

Arhitecturi nestructurate

Probleme

- **Noduri ale caror adrese IP sunt disponibile via NAT (cu restrictii)**
 - Nu pot fi servere TCP pentru rețeaua P2P
 - Soluție parțială: *reverse call*
 - A vrea să transfere de la B, iar B folosește NAT
 - A și B stabilesc conexiuni TCP cu serverul C (IP rutabil)
 - A poate cere lui B, via C, să realizeze o conexiune TCP de la B la A
 - A poate trimite o cerere lui B, iar B îi oferă răspunsul
 - Dacă A și B utilizează NAT?
- **Flash crowd: o creștere neașteptată de cereri pentru o anumită resursă**
 - Pentru conținutul dorit nu există suficiente copii încărcate
 - Cât timp ia unui utilizator să localizeze fișierul?
 - Câte mesaje va primi un nod datorită căutărilor realizate de alte noduri?
 - Se poate folosi un protocol de căutare generic, bazat pe TTL

Arhitecturi structurate

- Reprezinta solutia **academica** pentru P2P
- Scop:
 - Cautare cu succes
 - Timp de cautare in limite cunoscute
 - Scalabilitate demonstrata
- Abordare: **DHT (Distributed Hash Table)**
 - Se stocheaza perechi (key, value)
 - Key – nume de fisiere
 - Value – continut de fisier sau pointer la o locatie
 - Fiecare *peer* stocheaza o multime de (key, value)
 - *Operatii: gaseste nodul responsabil cu un Key*
 - Mapare *key – node*
 - Rutarea eficienta a cererilor de *insert/lookup/delete* asociate cu acest nod
 - Se permite o mare fluctuatie a nodurilor

Arhitecturi structurate

- Aspect de interes: **localizarea conținutului**
- Idee: Responsabilitatea este distribuita mai multor noduri ale rețelei de acoperire, într-un mod adaptiv
- Fiecarei resurse i se asociază o cheie unică via o funcție *hash*: $h(\text{"Curs Retele"}) \rightarrow 7929$; Intervalul de valori ale funcției *hash* se distribuie în rețeaua P2P
- Fiecare nod trebuie să "cunoască" locația macar a unei singure copii a fiecărei resurse pentru care funcția sa *hash* ia valori în intervalul lui
- Nodurile pot menține în cache-ul propriu o copie a fiecărei resurse pe care trebuie să o "cunoască"

Arhitecturi structurate

- Aspect de interes: **dirijarea**
- Pentru fiecare resursa, un nod ce “cunoaste” resursa trebuie sa fie accesat pe calea cea mai “scurta”
- Abordarile de sisteme P2P structurate difera prin strategia de dirijare
- Nodurile din sistem formeaza o structura de date distribuita care poate fi: inel, arbore, hypercub, skip list etc.
- Se ofera un API pentru tabelele distribuite de hash-uri (DHT – Distributed Hash Table)
 - Dand o cheie k , API-ul va returna adresa IP a nodului responsabil pentru valoarea cheii k

Arhitecturi structurate

- **Implementari**

- Chord [MIT]
- Pastry [Microsoft Research UK, Rice University]
- Tapestry [UC Berkeley]
- Content Addressable Network (CAN) [UC Berkeley]
- SkipNet [Microsoft Research US, Univ. of Washington]
- Kademlia [New York University]
- Viceroy [Israel, UC Berkeley]
- P-Grid [EPFL Switzerland]

Arhitecturi structurate

- **Slab structurate**
 - Nodurile pot estima ce noduri stocheaza resursele cautate
 - Se evita broadcasturile oarbe
 - Se foloseste o propagare in lant (*chain mode propagation*): fiecare nod ia decizii locale privitoare la care va fi nodul urmator interogat
 - Cautarea unui fisier presupune utilizarea unei chei si a unui mecanism de *timeout*
 - Exemplu: **Freenet**

[<https://en.wikipedia.org/wiki/Freenet>]

Instrumente

- “IPFS is the Distributed Web” - <https://ipfs.io/>
 - A peer-to-peer hypermedia protocol to make the web faster, safer, and more open



HTTP is inefficient and expensive

HTTP downloads a file from a single computer at a time, instead of getting pieces from multiple computers simultaneously. With video delivery, a P2P approach could save 60% in bandwidth costs.

IPFS makes it possible to distribute high volumes of data with high efficiency. And zero duplication means savings in storage.



Humanity's history is deleted daily

The average lifespan of a web page is 100 days. Remember GeoCities? The web doesn't anymore. It's not good enough for the primary medium of our era to be so fragile.

IPFS provides historic versioning (like git) and makes it simple to set up resilient networks for mirroring of data.

Instrumente

- “IPFS is the Distributed Web” - <https://ipfs.io/>
 - A peer-to-peer hypermedia protocol to make the web faster, safer, and more open



The web's centralization limits opportunity

The Internet has been one of the great equalizers in human history and a real accelerator of innovation. But the increasing consolidation of control is a threat to that.

IPFS remains true to the original vision of the open and flat web, but delivers the technology which makes that vision a reality.



Our apps are addicted to the backbone

Developing world. Offline. Natural disasters. Intermittent connections. All trivial compared to interplanetary networking. The networks we're using are so 20th Century. We can do better.

IPFS powers the creation of diversely resilient networks which enable persistent availability with or without Internet backbone connectivity.

- “ IPFS is the Distributed Web” - <https://ipfs.io/>



Ms. File McFileron

1643KB

IPFS HASH:
QmTwpJ2r5GZrAa25s3KZnmr7gPp4dW5
x79jWp7bdG



- <https://github.com/ipfs/papers/raw/master/ipfs-cap2pfs/ipfs-p2p-file-system.pdf>

Rezumat

- **Paradigma peer-to-peer(P2P)**
 - Preliminarii
 - Definitii
 - Caracterizare
 - Tipuri de aplicatii
 - Infrastructuri
 - Instrumente

Bibliografie

- P2P Networking and Applications, John F. Buford, Heather Yu, Eng Keong Lua, 2009, Elsevier
- <https://www.kirsle.net/skype-switched-to-the-msn-messenger-protocol>



Intrebari?

Intrebari?